

```

#include <iostream>
#include <cuda_runtime.h>
using namespace std;

__global__ void matrixMul(int *A, int *B, int *C, int rowsA, int
colsA, int colsB) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < rowsA && col < colsB) {
        int sum = 0;
        for (int k = 0; k < colsA; k++) {
            sum += A[row * colsA + k] * B[k * colsB + col];
        }
        C[row * colsB + col] = sum;
    }
}

int main() {
    int rowsA, colsA, rowsB, colsB;

    cout << "Enter rows and columns of Matrix A: ";
    cin >> rowsA >> colsA;

    cout << "Enter rows and columns of Matrix B: ";
    cin >> rowsB >> colsB;

    if (colsA != rowsB) {
        cout << "Matrix multiplication not possible!" << endl;
        return 0;
    }

    int sizeA = rowsA * colsA;
    int sizeB = rowsB * colsB;
    int sizeC = rowsA * colsB;

    int *h_A = new int[sizeA];
    int *h_B = new int[sizeB];
    int *h_C = new int[sizeC];

    cout << "Enter elements of Matrix A:\n";
    for (int i = 0; i < sizeA; i++) {
        cin >> h_A[i];
    }

    cout << "Enter elements of Matrix B:\n";
    for (int i = 0; i < sizeB; i++) {
        cin >> h_B[i];
    }

    int *d_A, *d_B, *d_C;
    cudaMalloc((void**)&d_A, sizeA * sizeof(int));
    cudaMalloc((void**)&d_B, sizeB * sizeof(int));
    cudaMalloc((void**)&d_C, sizeC * sizeof(int));
}

```

```

        cudaMemcpy(d_A, h_A, sizeA * sizeof(int),
cudaMemcpyHostToDevice);
        cudaMemcpy(d_B, h_B, sizeB * sizeof(int),
cudaMemcpyHostToDevice);

        dim3 blockSize(16, 16);
        dim3 gridSize((colsB + blockSize.x - 1) / blockSize.x,
                        (rowsA + blockSize.y - 1) / blockSize.y);

        matrixMul<<<gridSize, blockSize>>>(d_A, d_B, d_C, rowsA, colsA,
colsB);
        cudaDeviceSynchronize();

        cudaError_t err = cudaGetLastError();
        if (err != cudaSuccess) {
            cout << "CUDA Error: " << cudaGetErrorString(err) << endl;
            return 1;
        }

        cudaMemcpy(h_C, d_C, sizeC * sizeof(int),
cudaMemcpyDeviceToHost);

        cout << "\nResultant Matrix C:\n";
        for (int i = 0; i < rowsA; i++) {
            for (int j = 0; j < colsB; j++) {
                cout << h_C[i * colsB + j] << " ";
            }
            cout << endl;
        }

        cudaFree(d_A);
        cudaFree(d_B);
        cudaFree(d_C);

        delete[] h_A;
        delete[] h_B;
        delete[] h_C;

        return 0;
}

```